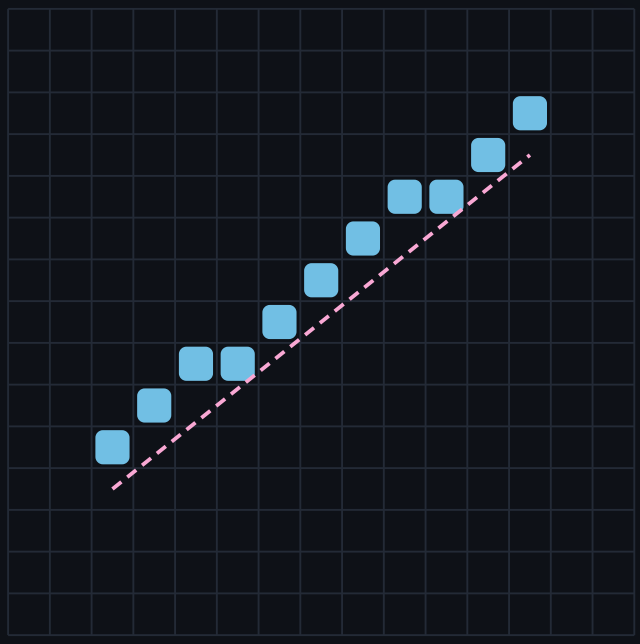
CMPN322 · DR. MANAR DONIA · MIDTERM

Computer Graphics & Man-Machine Interface

Study Guide — theory, solved problems and OpenGL code from Lectures 1–6, Sections 1–7 and Exercise Sheet 1. Every item cites the file and page it came from.

Raster & Color | Window → Viewport | DDA · Bresenham | Midpoint Circle · Ellipse | 2D / 3D Transformations | OpenGL / GLUT



Bresenham (20,10) → (30,18)



6	7	1	13
Lectures	Sections	Exercise sheet	Source files only

Contents

Theory · solved problems · code — from the 13 reference files only

01 **Basics of Computer Graphics**
Computer graphics vs image processing · models · systems · input primitives · vector vs raster · OpenGL pipeline
Lec 1 · SEC 1 and 2

02 **Pictures, Raster Images & Color**
Model to image · output primitives · polygons · raster images · frame buffer · direct coding & LUT
Lec 2 · Graphics_SEC 3

03 **Window to Viewport Mapping**
Window vs viewport · mapping equations · gluOrtho2D · multiple viewports · aspect ratio
Lec 3 · Graphics_Sec 4

04 **Line Drawing Algorithms**
DDA · Bresenham's midpoint line algorithm
Lec 4 · Graphics sec5

05 **Circle, Ellipse & Antialiasing**
Midpoint circle · midpoint ellipse · aliasing and super-sampling
Lec 5 · Graphics_Sec 6

06 **Geometric Transformations**
2D / 3D transformations · homogeneous coordinates · composite transformations
Lec 6 · Graphics_Sec 7

07 **OpenGL / GLUT Code**
The code given in the lecture slides
Lec 3 · Lec 4 · Lec 5

08 **Exercise Sheet 1**
Each question and where its answer is
Exercise Sheet 1

A **Sources**
Which part of the guide came from which file and page
All 13 files

How to read this guide

- Each card shows its source at the top right, e.g. `Source: Lec 2 · p35` = Lecture 2, page 35.
- Definitions, answers, tables and code are copied from the reference files as written. Solved tables keep the reference's numbers and rounding.
- Drawings are made from the numbers of the reference solution.
- Under every theory card, the yellow box ✍ **EXAM ANSWER** gives the likely exam question and the answer to write for the full mark.

صناديق **الفكرة** بالعربي بتعيد نفس كلام المرجع بشكل مختصر، ومش بتضيف معلومة من بره.

CG / LECTURE 1 · SEC 1 & 2

Basics of Computer Graphics

01

Computer graphics vs image processing · models · systems · input primitives · vector vs raster · OpenGL pipeline

T1

Source: Lec 1 · p6–7 · SEC 1 and 2 · p2

Computer graphics and image processing.

- Computer graphics are concerned with the **creation** of images.
- Image processing: analyze pictures to derive descriptions (often in mathematical or geometrical forms) of objects appeared in the pictures.
- Computer graphics means **creation, storage and manipulation** of models and images.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Differentiate between computer graphics and image processing.

Computer graphics	Image processing
Concerned with the **creation** of images	**Analyze** pictures to derive descriptions of objects appeared in the pictures
Model → picture	Picture → description (mathematical or geometrical forms)

Q: Define computer graphics.

- Computer graphics means **creation, storage and manipulation of models and images**.

الفكرة الـ computer graphics بيعمل الصورة (creation)، والـ image processing بيحلل الصورة عشان يطلّع منها وصف. وتعريف الـ CG: creation وstorage وmanipulation للـ models والصور.

T2

Source: Lec 1 · p7

Computer graphics is a cross-disciplinary field.

Computer graphics is a cross-disciplinary field in which physics, mathematics, human perception, human-computer interaction, engineering, graphic design, and art all play important roles.

- We use **physics** to model light and to perform simulations for animation.
- We use **mathematics** to describe shape and motion amount and direction.
- **Human perceptual abilities** determine our allocation of resources — we don't want to spend time rendering things that will not be noticed.
- We use **engineering** in optimizing the allocation of bandwidth, memory, and processor time.
- **Graphic design and art** combine with human-computer interaction to make the computer-to-human direction of communication most effective.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Why is computer graphics a cross-disciplinary field?

- **Physics** — to model light and to perform simulations for animation.
- **Mathematics** — to describe shape and motion.
- **Human perception** — not spending time rendering things that will not be noticed.
- **Engineering** — optimizing bandwidth, memory and processor time.
- **Graphic design and art + HCI** — effective computer-to-human communication.

الفكرة كل علم ليه دور: physics للضوء والـ animation، وmath للشكل والحركة، وhuman perception عشان منرسمش حاجة مش هتتلاحظ، وengineering لـ bandwidth والـ memory والـ processor time، والـ design والـ art عشان التواصل مع المستخدم.

T3

Source: Lec 1 · p9 · SEC 1 and 2 · p3

Computer graphics model.

- **Geometric model:** describes how objects appears (it defines attributes that describe color, texture or shape of the object).
- **Mathematical model:** is a model of a physical process in which it describes how object moves.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Differentiate between the geometric model and the mathematical model.

Geometric model	Mathematical model
Describes **how objects appears**	Model of a physical process: describes **how object moves**
Attributes: color, texture or shape	Motion of the object

الفكرة Geometric model = شكل الجسم (color وtexture). Mathematical model = حركة الجسم (shape).

T3b

Source: Lec 1 · p10

Computer graphics applications.

Computer graphics has number of applications:

- Video games
- Virtual reality
- Architectural design
- Scientific visualization
- Medical imaging

- Education

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: List the applications of computer graphics.

- Video games
- Virtual reality
- Architectural design
- Scientific visualization
- Medical imaging
- Education

T4

Source: Lec 1 · p11 · SEC 1 and 2 · p3

Computer graphics systems: non-interactive and interactive.

- **Non-interactive computer graphics:** the picture is produced on the monitor, and the user does not have any control over the image as shown in **animation movies**. It involves only **one-way communication** between the computer and the user where users can see the produced image, and cannot make any change in the image.
- **Interactive computer graphics:** the picture is produced on the monitor, and the user can make any change by editing modification with an input device. It involves **two-way communication** between the computer and the user as shown in **games**.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Compare non-interactive and interactive computer graphics.

Non-interactive	Interactive
User has **no control** over the image	User **can make any change** with an input device
One-way communication	**Two-way** communication
Example: **animation movies**	Example: **games**

الفكرة Non-interactive = اتجاه واحد، زي أفلام الـ animation. Interactive = اتجاهين، زي الـ games.

T5

Source: Lec 1 · p12 · SEC 1 and 2 · p1

Computer graphic system.

- **I/P device** are the h/w that is used to transfer I/P to the computer. The data can be in the form of text, graphics or sound.
- **Processor (CPU & GPU):** CPU gets the specifications for a geometric shape from input device and hands the geometric data (with additional operations) to the GPU. The GPU rasterizes the data by deciding which

pixels to use to represent the data, then places the pixel data into the framebuffer to show on the display screen.

- **Frame buffer:** it is a block of memory dedicated to graphics output which holds data representing all the pixels in a complete video frame.
- **O/P device** displays data from memory of the computer.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Explain the components of a computer graphics system.

- **I/P device:** hardware that transfers input (text, graphics or sound) to the computer.
- **CPU:** gets the specifications of a geometric shape from the input device and hands the geometric data to the GPU.
- **GPU:** rasterizes the data (decides which pixels represent it) and places the pixel data into the frame buffer.
- **Frame buffer:** a block of memory dedicated to graphics output which holds data representing all the pixels in a complete video frame.
- **O/P device:** displays data from memory of the computer.

الفكرة الـ CPU بياخد الـ geometric data من الـ input device وبديها للـ GPU، والـ GPU بيعمل rasterize وبحط الـ pixels في الـ frame buffer.

T6

Source: Lec 1 · p13–14 · SEC 1 and 2 · p1

Input primitives.

Different types of data are retrieved from input devices called **input primitives**.

Device	Input primitive
Locator devices (interactive drawing)	Coordinate position (x, y)
String devices (text input)	A string of characters
Valuator (games)	Scalar values (typically between 0 and 1) — can be used to fix the length of a line, the speed of an action, or perhaps the size of a picture
Choice (interactive menu selection)	A selection from a list of options
Pick (identify a portion of a picture for further processing)	Selection of a part of the screen

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Define input primitives and list the input devices.

- **Input primitives:** the different types of data retrieved from input devices.

- Locator → (x, y) · String → characters · Valuator → scalar 0–1 · Choice → selection from a list · Pick → part of the screen.

الفكرة Locator ← (x, y)، String ← حروف، Valuator ← رقم بين 0 و1، Choice ← اختيار من list، Pick ← جزء من الشاشة.

T7

Source: Lec 1 · p15–16 · SEC 1 and 2 · p4

Graphics display device: vector and raster.

- **Vector displays** construct pictures by drawing straight-line segments between specified endpoint positions. Picture information is then stored as a set of line-drawing instructions. It is best for printing since it is composed of a series of mathematical curves; vector graphics print precisely even when they are enlarged. It's known as scalable vector graphics (SVG).
- **Raster displays** use a frame buffer to store the color value for each screen position (pixel). The larger the image, the more disk space the image file will take up (a 200 × 200 image requires less space than a 500 × 500 one). Scaling down these images is easy but enlarging a bitmap makes it pixelated or simply blurred.

Raster	Vector
Composed of pixels	Composed of connected sequence of points (paths)
It produces large file size	It produces small file size
It provides more details shape	It provides smooth shape but limited to set of colors
Used for details images such as photo	Used for less details images such as logos

If your project requires scalable shapes and solid colors, vector is the best choice as for logos and icons; if your project requires complex color blends, raster is the preferred format as for digital photos.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Differentiate between raster and vector displays.

Raster	Vector
Frame buffer stores the color of each pixel	Draws straight-line segments between endpoints
Composed of pixels	Connected sequence of points (paths)
Large file size	Small file size
More details shape	Smooth shape, limited set of colors
Enlarging → pixelated / blurred	Prints precisely when enlarged (SVG)
Photos (complex color blends)	Logos and icons (solid colors)

الفكرة Vector: paths، حجم صغير، بيتكبّر من غير ما يبوظ، للـ logos والـ icons. Raster: pixels، حجم كبير،

بيتشوّه لما يتكبّر، للصور (photos).

T8

Source: Lec 1 · p17 · SEC 1 and 2 · p2

Conceptual framework of computer graphics.

- Interactive computer graphics consists of two major components at two levels:
- **Hardware level:** the computer receives input from interactive devices and shows images on display devices.
- **Software level:** it involves three components — **Application program** creates, stores into and retrieves from the application model such as Word, Excel, Paint or Photoshop (handles user's input). **Application model** captures all the data or objects to be pictured on the screen, that stored in the application database. **Graphics library** or application programmer's interface (API) is a set of procedures for handling graphics such as OpenGL, Direct3D or Java3D; it is intermediary between application and display hardware (graphics system).

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Explain the conceptual framework of computer graphics.

- Two major components at two levels:
- **Hardware level:** receives input from interactive devices and shows images on display devices.
- **Software level: application program** (creates, stores into and retrieves from the application model, handles user's input) · **application model** (all data/objects to be pictured, in the application database) · **graphics library (API)** (procedures for handling graphics, e.g. OpenGL; intermediary between application and display hardware).

الفكرة مستويين: hardware وsoftware. والـ software فيه 3 حاجات: application program وapplication model وgraphics library (API).

T9

Source: Lec 1 · p18–21

2D first, OpenGL and coordinate systems.

- 2D is a good stepping stone towards 3D; many issues are much easier to understand in 2D; no need to simulate lights, cameras, etc.; 2D is still important and the most common use of computer graphics.
- OpenGL stands for Open Graphics Library, producing 2D and 3D graphics; it supports every major operating system and a wide variety of programming languages, but the primary interface of OpenGL is in C++.
- **Cartesian coordinates** in math, engineering: modeled as floating point, X increasing right, Y increasing up.
- **Display (physical) coordinates:** integer values only, X increasing right, Y increasing down by 1 unit = 1 pixel.

- We specify a display window with a width of 400 pixels and a height of 300 pixels using `glutInitWindowSize(400, 300)` ; `glutInitWindowPosition(50, 100)` gives an initial location for the upper left corner of the display window; then specify the region of Cartesian space that occupies the window using `gluOrtho2D(xmin, xmax, ymin, ymax)` . This region is known as the **clipping region**.
- OpenGL uses normalized device coordinates: the coordinates of each vertex should be between −1.0 and 1.0.

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: Differentiate between Cartesian and display coordinates.

Cartesian	Display (physical)
Floating point	Integer values only
X right, Y up	X right, Y down, 1 unit = 1 pixel

Q: What does each GLUT function do?

- `glutInitWindowSize(400, 300)` : window 400 × 300 pixels.
- `glutInitWindowPosition(50, 100)` : initial location of the **upper left corner** of the display window.
- `gluOrtho2D(xmin, xmax, ymin, ymax)` : the **clipping region**.
- OpenGL uses normalized device coordinates: every vertex between −1.0 and 1.0.

الفكرة Cartesian: الـ y لفوق والأرقام float. Display: الـ y لتحت والأرقام integer والوحدة = pixel.
وgluOrtho2D بيحدد الـ clipping region.

T9b

Source: Lec 1 · p18

2D or 3D first.

- 2D is a good steppingstone towards 3D; many issues are much easier to understand in 2D — you can't be a 3D animator unless you have learned some 2D first.
- No need to simulate lights, cameras, etc.
- Intro to modelling (SketchUp) vs rendering (use colors, materials, and textures to create a realistic image of the model).
- 2D is still important and the most common use of computer graphics as in advertising and many other fields.

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: Why do we learn 2D before 3D?

- 2D is a good steppingstone towards 3D; many issues are easier to understand in 2D.
- No need to simulate lights, cameras, etc.
- 2D is still the most common use of computer graphics (advertising).

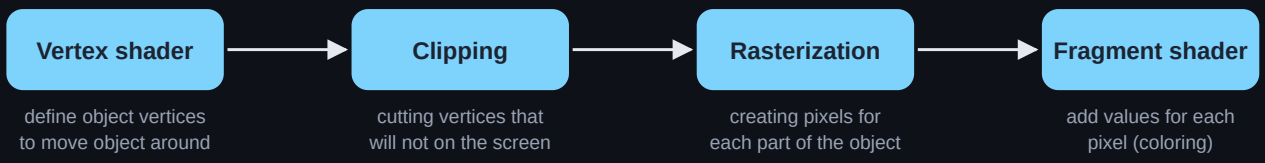
T10

Source: Lec 1 · p22–24 · SEC 1 and 2 · p2

Transformations and the OpenGL pipeline.

- Geometric transformations in 2D (note the z-coordinate is 0): translate, rotate, scale. Positive angles rotate counter-clockwise, here about the origin.
- Transformations can be composed (combined by multiplying matrices) but are NOT commutative, so proper order is vital.
- **Graphics pipeline** is a conceptual model that describes what steps a graphics system needs to perform to render a 3D scene to a 2D screen.

OpenGL pipeline



define object vertices to move object around

cutting vertices that will not on the screen

creating pixels for each part of the object

add values for each pixel (coloring)

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Define the graphics pipeline and explain its stages.

- A **conceptual model** that describes what steps a graphics system needs to perform to render a **3D scene to a 2D screen**.
- **Vertex shader** → define vertices to move the object (translation, scaling, rotation).
- **Clipping** → cut vertices that will not be on the screen.
- **Rasterization** → create pixels for each part of the object.
- **Fragment shader** → add values for each pixel (coloring).

Q: Why is the order of transformations important?

- Transformations are composed by multiplying matrices but are **NOT commutative**, so proper order is vital.

الفكرة الـ pipeline بالترتيب: vertex shader ← clipping ← rasterization ← fragment shader. والـ transformations مش commutative، فالترتيب مهم.

T11

Source: Lec 1 · p25–26

Textbooks.

- Computer Graphics Using OpenGL — F. S. Hill, Jr. and Stephen M. Kelley, Jr.
- Computer Graphics with OpenGL, 4th Edition — Donald D. Hearn, M. Pauline Baker, Warren Carithers (Pearson).

CG / LECTURE 2 · SEC 3

Pictures, Raster Images & Color

02

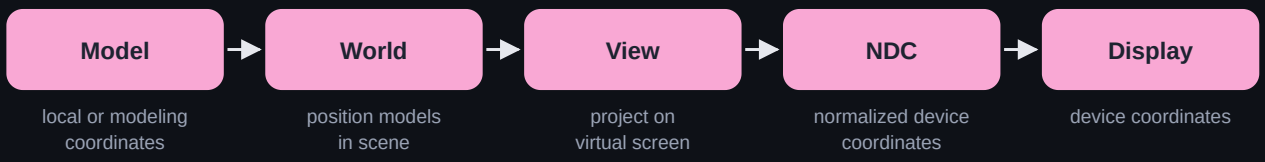
Model to image · output primitives · polygons · raster images · frame buffer · direct coding & LUT

T1

Source: Lec 2 · p3–9 · Graphics_SEC 3 · p1

From model to image.

Coordinates and transformations



1. Object shapes are defined within their own modeling-coordinate systems.
2. These shapes are then placed within a global world-coordinate scene.
3. The world-coordinate data is processed through the viewing pipeline, converting it into viewing and projection coordinates, and further into normalized coordinates.
4. Finally, device drivers take the normalized-coordinate representation of the scene and send it to the output devices for display.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Explain the steps from model to image.

1. Model coordinates → World → View → NDC → Display (device).
2. Shapes defined in their own **modeling** coordinates → placed in a global **world** scene → **viewing and projection** coordinates → **normalized** coordinates → device drivers send them to the **output device**.

الفكرة الترتيب: Model ← World ← View ← NDC ← Display.

T2

Source: Lec 2 · p10–17

Elements of pictures created in computer graphics.

- **Output primitives:** the basic geometric structure used to describe scenes — polyline, text, filled regions, raster images. Each element has a set of attributes: the characteristics that affect how it appears, such as color, style, and thickness.
- **Polylines:** connected sequence of straight lines; pictures made up of polylines (line drawing); each line is called an edge; two adjacent lines meet at a vertex. A special case is a line segment that shrinks to a single

point: `drawDot(x1, y1)`.

- Attributes of lines and polylines: color, thickness, type (dashed, solid, dotted), way edges are blended (non-joint, mitered, rounded, beveled).
- **Text:** `drawString(x, y, string)` ; attributes: font face, weight, size, style, spacing, orientation.
- **Filled region:** shape filled with color; boundary of the filled shape is a polygon: `fillPolygon(poly, pattern)` ; attributes: enclosing border, pattern, color.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: List the output primitives and their attributes.

- **Output primitives:** polyline, text, filled regions, raster images.
- **Attributes:** the characteristics that affect how a primitive appears (color, style, thickness).
- Line: color, thickness, type, edge blending · Text: font face, weight, size, style, spacing, orientation · Filled region: enclosing border, pattern, color.

الفكرة الـ output primitives أربعة: polyline وtext وfilled regions وraster images، ولكل واحد زي attributes color وstyle وthickness.

T2b

Source: Lec 2 · p11, p16

Polyline pictures and character shapes.

- Polyline: the pictures made up of polylines are called **line drawing** — e.g. a polyline drawing of a dinosaur, a plot of a mathematical function, a wireframe rendering of a 3D object.
- A character shape is defined by a polyline or by an arrangement of dots.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: What is a line drawing?

- Pictures made up of polylines — e.g. a polyline drawing of a dinosaur, a plot of a mathematical function, a wireframe of a 3D object.

T3

Source: Lec 2 · p12 · Graphics_SEC 3 · p1

Polygon.

- Polygon is a closed polyline where its start point equals its last point.
- **Simple polygon** has well-defined two regions (inside and outside).
- **Complex polygon** has self-intersecting sides.
- **Concave polygon**: at least one diagonal lies outside the polygon (Graphics_SEC 3); concave polygon if it is curved inward (Lec 2).

- **Convex polygon**: all its diagonals lie inside the polygon (Graphics_SEC 3); convex polygon if it is curved outward (Lec 2).

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Define the polygon and its types.

Polygon	Meaning
Polygon	Closed polyline: start point = last point
Simple	Two well defined regions (inside / outside)
Complex	Self-intersecting sides
Concave	At least one diagonal outside (curved inward)
Convex	All diagonals inside (curved outward)

الفكرة Concave: فيه diagonal واحد على الأقل برّه الـ polygon (curved inward). Convex: كل الـ diagonals جوّه (curved outward).

T4

Source: Lec 2 · p18–24 · Graphics_SEC 3 · p1

Raster images.

- Image is made up of many cells (pixels) in different shades; stored as a 2D array of numerical values; the array as a whole is often called a "pixel map" or "bitmap".
- How raster images are created: **hand designed** images (paint program), **computed** images (an algorithm renders a scene), **scanned** images.
- **Monochrome** raster images (bi-level): pixels contain one of two values.
- **Gray-scale**: pixel depth = number of bits needed to represent gray level; N bits has 2^n values (2 bits → 4, 4 bits → 16, 8 bits → 256 gray levels).
- **Color** raster images: pixel contains a triple of red, green and blue; color depth of 3 allows one bit for each color; true color image has a color depth of 24 bits (8 bits for each color).

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Define the raster image and how it is created.

- Image made of many cells (**pixels**) stored as a 2D array of values — called a **pixel map** or **bitmap**.
- Created by: hand designed images (paint program), computed images (an algorithm renders the scene), scanned images (digitized photograph).
- Types: monochrome (bi-level), gray-scale (N bits → 2^N levels), color (RGB; true color = 24 bit).

الفكرة N bits ← 2^N gray levels. والـ true color = 24 bit، يعني 8 bits لكل لون.

T5

Source: Lec 2 · p25–30

Raster displays, flat panels, frame rate vs refresh rate.

- **Framebuffer** is a portion of memory containing data representing all the pixels in a complete video frame. **Pixel:** one dot of the raster. **Scan line:** a row of pixels.
- **Emissive displays** convert electrical energy into light (LED). **Non-emissive displays** use optical effects to convert light from some other source into graphics patterns (LCD).
- **Frame rate (FPS):** how many unique, complete images your GPU can create and send to the monitor each second. **Refresh rate (Hz):** how many times per second your monitor can physically update the image it displays.
- GPU frame rate < monitor refresh rate → the monitor displays the old frame again → **stuttering**. GPU frame rate > refresh rate → **screen tearing**. Therefore, you should match the two features.
- **Raster scan system:** the scan controller causes the frame buffer to "send" each pixel through a converter to the appropriate physical spot; the converter takes a pixel value such as 01001011 and converts it to the corresponding quantity that produces a spot of color.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Compare frame rate and refresh rate.

Frame rate (FPS)	Refresh rate (Hz)
Images the GPU creates per second	Updates per second the monitor can show
Variable (device power)	Fixed maximum (monitor hardware)
FPS < Hz → stuttering	FPS > Hz → screen tearing

Q: Differentiate between emissive and non-emissive displays.

Emissive	Non-emissive
Convert electrical energy into light	Use optical effects on light from another source
LED	LCD

الفكرة لو الـ FPS أقل من الـ Hz ← stuttering. لو أكتر ← screen tearing. فلازم الاتنين يتساووا.

T5b

Source: Lec 2 · p26, p29–30

Graphics display devices and raster scan displays.

- The display device is an output device used to represent the information in visual form. Most displays used today for computer graphics are raster displays; the most familiar are flat panels, which include: smartphones and tablets, smart TVs, digital signage, automotive infotainment screens.
- Raster devices have a display surface on which the image is presented; the display surface has a certain number of pixels that it can show, such as 480 rows, where each row contains 640 pixels.

- In old raster scan systems, the electron beam is swept across the screen one row at a time.
- Nowadays, instead of an electron beam, the entire frame of pixel data is sent from the graphics card to the monitor digitally (via HDMI, DisplayPort, etc.) as a continuous stream, row by row — the direct digital descendant of the old raster scan.
- Raster scan system: the graphics program is stored in system memory and is executed by the CPU; the framebuffer contains all the pixels intensity values in a complete video frame.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Explain the raster scan display.

- The display surface has a certain number of pixels (e.g. 480 rows × 640 pixels).
- Old: the electron beam is swept across the screen one row at a time.
- Now: the whole frame is sent digitally (HDMI, DisplayPort) row by row.
- The scan controller makes the frame buffer send each pixel through a converter to its spot on the screen.

T6

Source: Lec 2 · p33–34

Video monitors.

For a simple color, frame buffer is implemented with 3 bit planes, one for each primary color (red, green, blue); each bit plane drives an individual color gun. For better resolution, additional bit planes can be used for each color: $2^2 \times 2^2 \times 2^2 = 2^6 = 64$ different colors.

Red	Green	Blue	Color
0	0	0	Black
0	0	1	Blue
0	1	0	Green
0	1	1	Cyan
1	0	0	Red
1	0	1	Magenta
1	1	0	Yellow
1	1	1	White

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: How many colors with bit planes?

- 3 bit planes (1 per color) → 2^3 = **8 colors**; each bit plane drives an individual color gun.
- 2 planes per color → 2^6 = **64 colors**.

الفكرة 3 bit planes ← 8 ألوان. 2 planes لكل لون ← 2^6 = 64 لون.

T7

Source: Lec 2 · p35–38 · Graphics_SEC 3 · p1

Pixel color representation (color palette).

- **Color palette** is the set of possible colors displayed by the system.
- **1. Direct coding** represents each pixel's color directly. Frame buffer with 6 planes (2 for each color) → 4 levels each → 2^6 = 64 colors. True color (24 planes, 8 for each color) → 2^{24} = 16 million colors.
- **2. Indexed color and LookUp Table (LUT)** offers a programmable association between pixel value and final color. LUT is a compromise between our desire to have a lower storage requirement and our need to support enough number of colors. Each image pixel does not contain the full specification of its color, but only its index into the palette.
- `setPalette(39, 17, 25, 4)` then `drawDot(479, 532, 39)`. LUT of $(2^8 \times 24)/8$ = 768 bytes.

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: Differentiate between direct coding and LUT.

Direct coding	LUT (indexed color)
Pixel holds its full color	Pixel holds only an index into the palette
e.g. 24 bits → 16 million colors	Colors in a separate color lookup table
More memory	Compromise: lower storage + enough colors

Q: Laws.

- Colors (direct) = $2^n \times 2^n \times 2^n$
- LUT entries = 2^(pixel bits) · LUT size = entries × width / 8 bytes
- Frame buffer = W × H × bits / 8 bytes

الفكرة Direct coding: الـ pixel فيه اللون كله. LUT: الـ pixel فيه index بس في الـ palette.

T7b

Source: Lec 2 · p38

Expensive vs inexpensive system (direct coding vs LUT).

	System #1 (expensive)	System #2 (inexpensive)
Frame buffer	24 bit/pixel, no LUT	8 bit/pixel along with a LUT 24 bits wide
Colors	Each of its 1.3 million pixels can be set to any one of 2^{24} colors; eight bits drive each of the DAC's	Can display 2^{24} different colors, but only 256 at a time

Memory	1024 × 1280 × 24 bits — almost 4 megabytes	1024 × 1280 × 8 — about 1 megabyte; the LUT requires a trivial 768 bytes: (2^8 × 24)/8 = 768 bytes
Cost	Expensive	Much less expensive than system #1 if memory costs a significant amount per megabyte

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: Why is System #2 less expensive than System #1?

- System #1: 24 bit/pixel, no LUT → 1024 × 1280 × 24 bits ≈ 4 MB.
- System #2: 8 bit/pixel + 24-bit LUT → ≈ 1 MB + 768 bytes; 2^{24} colors but only 256 at a time.

P1

Source:

Problems: color palette and LUT.

Problem	Solution	Source
Direct coding of RGB with 3 bits per primary color: how many colors?	$2^3 \times 2^3 \times 2^3$ = **512** color	Lec 2 · p39
10 bits pixel value in a 24-bit lookup table: entries and size?	entries = 2^{10} = **1024** entry · size = (2^{10} × 24)/8 = **3072 byte**	Lec 2 · p39
Direct coding with 5 bits per primary color: how many colors?	2^5 = 32 levels each → 32 × 32 × 32 = **32,768** colors	Graphics_SEC 3 · p1
10-bit pixel value in a 32-bit LUT: entries and size?	2^{10} = **1,024** entries · (1024 × 32)/8 = **4,096 bytes** (4 KB)	Graphics_SEC 3 · p1
Frame buffer with 12 planes (4 for each color): colors in the palette?	2^4 = 16 levels each → 16 × 16 × 16 = **4,096** colors	Graphics_SEC 3 · p1

الفكرة عدد الألوان = 2 أس عدد الـ bits. عدد الـ LUT entries = 2 أس الـ bits الـ pixel. حجم الـ LUT = entries × عرض الـ LUT ÷ 8.

P2

Source: Graphics_SEC 3 · p2

Compare System A (32 bits/pixel frame buffer, no LUT) and System B (12 bits/pixel frame buffer with a 30-bit LUT) for a 1920 × 1080 display.

- **System A:** frame buffer = 1920 × 1080 × 32 bits = 66,355,200 / 8 = 8,294,400 bytes ≈ **8.29 MB**.
- **System B:** frame buffer = 1920 × 1080 × 12 bits = 24,883,200 bits = 3,110,400 bytes ≈ 3.11 MB · LUT entries = 2^{12} = 4,096 · LUT size = (2^{12} × 30)/8 = 15,360 bytes ≈ 0.015 MB · total ≈ 3.11 + 0.015 = **3.125 MB**.

الفكرة حجم الـ frame buffer = العرض × الطول × 8 ÷ bits per pixel.

P3

Source: Graphics_SEC 3 · p2

A display system uses a frame buffer with 10 bits per pixel and a LUT with 36-bit entries; resolution 2560 × 1440.

- (a) Simultaneous colors: 10-bit pixel value → 2^{10} = **1,024** colors.
- (b) Total palette colors: 36-bit LUT → 2^{36} = **68,719,476,736** colors (≈ 68.7 billion).
- (c) Frame buffer = 2560 × 1440 × 10 bits = 36,864,000 bits = 4,608,000 bytes ≈ 4.39 MB · LUT = (2^{10} × 36)/8 = 4,608 bytes ≈ 0.0044 MB · total ≈ **4.394 MB**.
- (d) Direct coding with 10 bits per primary: 30 bits/pixel → 2560 × 1440 × 30 bits = 110,592,000 bits = 13,824,000 bytes ≈ **13.18 MB**.

CG / LECTURE 3 · SEC 4

Window to Viewport Mapping

03

Window vs viewport · mapping equations · gluOrtho2D · multiple viewports · aspect ratio

T1

Source: Lec 3 · p2–3 · Graphics_Sec 4 · p1

Two-dimensional viewing transformation.

- The two-dimensional viewing transformation maps world-coordinate scenes to device coordinates, often referred to as the **window-to-viewport transformation**. This process primarily involves converting clipping-window coordinates to viewport coordinates.
- Once a world-coordinate scene has been constructed, we could set up a separate two-dimensional, viewing-coordinate reference frame for specifying the clipping window.
- To ensure device independence, graphics systems convert object descriptions into normalized coordinates (ranging from 0 to 1 or −1 to 1).
- **Window to viewport mapping** is the process of mapping or transforming a 2D world coordinate scene to device coordinates. **Window is what to be displayed; viewport is where to be displayed.**

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Define the window to viewport mapping.

- The process of mapping or transforming a 2D world coordinate scene to device coordinates.
- **Window** = what to be displayed · **Viewport** = where to be displayed.

الفكرة Window = إيه اللي يتعرض. Viewport = فين يتعرض.

T2

Source: Graphics_Sec 4 · p1

Window and viewport (Sec 4).

- A **window** is a rectangular region in world coordinates that defines what part of the scene is selected for display. Defined by: xwmin, xwmax, ywmin, ywmax.
- A **viewport** is a rectangular region on the display device that defines where on the screen the selected scene is drawn. Defined by: xvmin, xvmax, yvmin, yvmax.
- The window-to-viewport transformation is necessary for device independence as the same scene description can be rendered on a 640×480 monitor, a 4K display, or a printer by simply changing the viewport — no change to the model or window is needed. Also different viewports on the same screen can show different windows of the same scene (e.g., a CAD program showing top, front, and side views).

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Why is the window-to-viewport transformation necessary?

- For **device independence**: the same scene is rendered on any device (640×480, 4K, printer) by only changing the viewport.
- Different viewports can show different windows of the same scene.

الفكرة تغيير الـ viewport بس بيخلي نفس الـ scene تتعرض على أي جهاز من غير ما نغيّر الـ model أو الـ window.

T3

Source: Lec 3 · p5 · Graphics_Sec 4 · p1

Window to viewport mapping equations.

Suppose we have a point (XW, YW) and we want to find the corresponding viewport point (XV, YV). If the relative position is the same:

```
(XW − XWmin)/(XWmax − XWmin) = (XV − XVmin)/(XVmax − XVmin) · (YW − YWmin)/(YWmax − YWmin) = (YV − YVmin)/(YVmax − YVmin)
Sx = (XVmax − XVmin)/(XWmax − XWmin) · Sy = (YVmax − YVmin)/(YWmax − YWmin)
XV = XVmin + Sx·(XW − XWmin) · YV = YVmin + Sy·(YW − YWmin)
```

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: Write the mapping equations.

- Sx = (XVmax − XVmin)/(XWmax − XWmin) · Sy = (YVmax − YVmin)/(YWmax − YWmin)
- XV = XVmin + Sx·(XW − XWmin) · YV = YVmin + Sy·(YW − YWmin)

الفكرة احسب Sx وSy الأول، وبعدين عوّض في XV وYV.

T3b

Source: Lec 3 · p4, p8

Doing it in OpenGL.

- As the size of the window is greater than the size of the viewport, we need to decrease the size of the window according to the size of the viewport (mathematical computations map window and viewport).
- OpenGL makes it very easy to use the window to viewport mapping: it automatically passes each vertex it is given (via a glVertex2*() command) through a sequence of transformations that carry out the desired mapping. It also automatically clips off parts of objects lying outside the world window.
- For 2D drawing the world window is set by `gluOrtho2D(left, right, bottom, top)` — lower left corner (left, bottom), upper right corner (right, top) — and the viewport is set by `glViewport(x, y, width, height)` — lower left corner (x, y), upper right corner (x + width, y + height).

- **By default the viewport is the entire screen window:** if W and H are the width and height of the screen window, the default viewport has lower left corner at (0, 0) and upper right corner at (W, H).

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: Which functions set the window and the viewport in OpenGL?

- `gluOrtho2D(left, right, bottom, top)` → world window.
- `glViewport(x, y, width, height)` → viewport.
- Default viewport = entire screen window: (0, 0) to (W, H).

P1

Source:

Problems: window to viewport.

Given	Answer	Source
Window (20, 80, 40, 80), viewport (30, 60, 40, 60), (XW, YW) = (30, 80)	Sx = (60 − 30)/(80 − 20) = 0.5 · Sy = 0.5 · XV = 30 + (0.5·(30 − 20)) = **35** · YV = 40 + (0.5·(80 − 40)) = **60**	Lec 3 · p6
Window (0, 2, 0, 1), viewport (40, 400, 60, 300), (XW, YW) = (1, 0.5)	Sx = (400 − 40)/(2 − 0) = 180 · Sy = (300 − 60)/(1 − 0) = 240 · XV = 40 + (180·(1 − 0)) = **220** · YV = 60 + (240·(0.5 − 0))	Lec 3 · p7
Same window and viewport, (XW, YW) = (1.5, 0.8)	XV = 40 + (180·(1.5 − 0)) = **310** · YV = 60 + (240·(0.8 − 0)) = **252**	Lec 3 · p9
Window 1.0 ≤ x ≤ 2.0, 0.1 ≤ y ≤ 1.1; viewport 0 ≤ X ≤ 200, 100 ≤ Y ≤ 300; world point (1.1, 0.8)	XV = XVmin + Sx·(XW − XWmin), YV = YVmin + Sy·(YW − YWmin) → (XV, YV) = **(20, 240)**	Graphics_Sec 4 · p1
Window (0, 10, 0, 5), viewport (50, 250, 100, 200); viewport point (150, 150) → (XW, YW)?	(XW, YW) = **(5, 2.5)**	Graphics_Sec 4 · p2

Lec 3 · p6



Lec 3 · p9



Graphics_Sec 4 · p1



Graphics_Sec 4 · p2



T4

Source: Lec 3 · p10–17

Display-window management with gluOrtho2D (the lecture's triangle).

DISPLAY-WINDOW MANAGEMENT: `glutInitWindowSize(400, 300)` , `glutInitWindowPosition(100, 150)` , triangle (−200, 10), (10, 10), (10, 150) with these settings of `gluOrtho2D` :



Can I draw a simple figure without gluOrtho2D? With `//gluOrtho2D(-100,100,-100,100);` commented out, the triangle (−200, 10), (10, 10), (10, 150) is not displayed, while the triangle (−0.5, 0), (0, 0.5), (0.5, 0) is displayed.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: What happens without gluOrtho2D?

- Coordinates are between −1 and 1: the triangle (−0.5, 0), (0, 0.5), (0.5, 0) is displayed, the triangle with large coordinates is not.
- Changing the order of the values in gluOrtho2D flips the image.

الفكرة ترتيب الأرقام في gluOrtho2D بيقلب الصورة. ومن غير gluOrtho2D، الإحداثيات بتبقى من 1− لـ 1.

T5

Source: Lec 3 · p18–20

Create multiple viewports where each viewport has a different scene.

Each `glViewport(x, y, width, height)` before `glBegin` draws the triangle in a different place: `glViewport(0, 0, 100, 100)`, `glViewport(100, 100, 100, 100)`, `glViewport(200, 200, 100, 100)`. In the last version a different `gluOrtho2D` is set before each viewport (the middle one uses `gluOrtho2D(-200, 200, 150, -150)`, so its triangle appears upside down).

Code: section 07, C2.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: How do we draw the scene in many viewports?

- Call `glViewport(x, y, width, height)` before each `glBegin` — each call draws the scene in a different place.

الفكرة كل glViewport قبل الرسم بيرسم الـ scene في مكان مختلف من الشاشة.

T6

Source: Lec 3 · p21–27 · Graphics_Sec 4 · p2

Setting the window and viewport automatically.

- **Setting window:** often the programmer does not know where or how big the object of interest lies in world coordinates. The usual approach is to find a window that includes the entire object: the object's **extent**

(bounding box) must be found — the aligned rectangle that just covers it.

- **Automatic setting of the viewport to preserve aspect ratio:** to draw the largest undistorted version of a figure that will fit in the screen window, specify a viewport that has the same aspect ratio as the world window. R = aspect ratio of world.
- If R > W/H: the viewport will have width W and height W/R → `glViewport(0, 0, W, W/R)`.
- If R < W/H: the viewport will have width H·R and height H → `glViewport(0, 0, H*R, H)`.

Largest undistorted viewport (screen window W × H, world aspect ratio R)



R > W/H → glViewport(0,0,W,W/R)



R < W/H → glViewport(0,0,H*R,H)

Example	Answer	Source
R = 1.6, screen window width 360, height 200	W/H = 360/200 = 1.8 → R < W/H → Hv = H = 200, Wv = H·R = 200 × 1.6 = 320 → `glViewport(0,0,320,200)`	Lec 3 · p26
R = 2, screen window width 360, height 200	W/H = 1.8 → R > W/H → Wv = W = 360, Hv = W/R = 360/2 = 180 → `glViewport(0,0,360,180)`	Lec 3 · p27
R = 1.5, screen window width 400, height 300	Screen aspect ratio = 400/300 = 1.333 → VW = 400, VH = W/R = 400/1.5 = 267	Graphics_Sec 4 · p2







✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: How to preserve the aspect ratio?

- R = aspect ratio of the world window; compare with W/H.
- R > W/H → `glViewport(0, 0, W, W/R)`
- R < W/H → `glViewport(0, 0, H*R, H)`
- Setting the window automatically: find the object's extent (bounding box).

الفكرة لو R > W/H ← العرض W والطول W/R. لو W/H > R ← الطول H والعرض H·R.

CG / LECTURE 4 · SEC 5

Line Drawing Algorithms

04

DDA · Bresenham's midpoint line algorithm

T1

Source: Lec 4 · p3–13

Basic elements and line drawing.

- Three basic elements: **points**, **lines**, **vectors**. Points are associated with locations in space; the only characteristic that distinguishes one point from another is its position. A line is a one-dimensional shape. Vectors have magnitude and direction; they represent displacements between points, or directions.
- `setPixel` stores the current color setting into the frame buffer at integer position (x, y); `getPixel` retrieves the current frame-buffer setting for a pixel location.
- To display the line on a raster monitor, the graphics system must first project the endpoints to integer screen coordinates and determine the nearest pixel positions along the line path between the two endpoints; then the line color is loaded into the frame buffer; reading from the frame buffer, the video controller plots the screen pixels.
- Idea: a line is sampled at unit intervals in one coordinate and the corresponding integer values nearest the line path are determined for the other coordinate.
- Trivial situations (do not need line-drawing algorithms): m = 0 horizontal line; m = ∞ vertical line; m = ±1 line y = ±x.
- y = m·x + b, m = (yend − y0)/(xend − x0), b = y0 − m·x0. It works for lines with a slope of 1 or less, but doesn't work well for lines with slope greater than 1. Solution: use symmetry.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: How does a raster system display a line?

1. Project the endpoints to integer screen coordinates and find the nearest pixels along the path.
2. Load the line color into the frame buffer.
3. The video controller reads the frame buffer and plots the pixels.

Q: Sampling and trivial lines.

- **Sampling:** measuring the values of the function at equal intervals.
- m = 0 horizontal · m = ∞ vertical · m = 1 → y = x (no algorithm needed).

الفكرة الخط بيتعمله sampling على coordinate واحد بخطوة 1، والـ coordinate التاني بيتقرّب لأقرب integer.

T1b

Source: Lec 4 · p9

Towards the ideal line.

- We can only do a discrete approximation: illuminate pixels as close to the true path as possible (consider bi-level display only — pixels are either lit or not lit).
- In the raster line algorithm we sample at unit intervals and determine the closest pixel position to the specified line path at each step.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: What is the ideal line on a raster display?

- Only a discrete approximation: illuminate pixels as close to the true path as possible (pixels lit or not lit).

T2

Source: Lec 4 · p15–17, p24 · Graphics sec5 · p1

DDA algorithm.

- The digital differential analyser (DDA) is a scan-conversion line algorithm based on using δx or δy.
- If m ≤ 1 (dx > dy): sample at unit x intervals (dx = 1): $x_{k+1} = x_k + 1$, $y_{k+1} = y_k + m$; round y to nearest integer value.
- If m > 1 (dy > dx): sample at unit y intervals (dy = 1): $y_{k+1} = y_k + 1$, $x_{k+1} = x_k + 1/m$; round x to nearest integer value.
- **Advantages:** it is a simple algorithm to implement; it is a faster algorithm than the direct line equation.
- **Disadvantages:** the floating-point arithmetic implementation of the DDA is time-consuming; the method of round-off is also time-consuming; the accumulation of round-off error in successive additions of the floating-point increment can cause the calculated pixel positions to drift away from the true line path for long line segments.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Explain the DDA algorithm.

- Sample at unit intervals in one coordinate and find the nearest integer for the other.
- |m| ≤ 1: x + 1, y + m · |m| > 1: y + 1, x + 1/m · round each value.
- **Advantages:** simple, faster than the direct line equation.
- **Disadvantages:** floating-point and round-off are time-consuming; round-off error accumulates → pixels drift from the true path for long lines.

الفكرة if m ≤ 1 → x + 1 و if m > 1. y + m ← y + 1 و x + 1/m. عيوبه: الـ floating point والـ round-off error.

P1

Source:

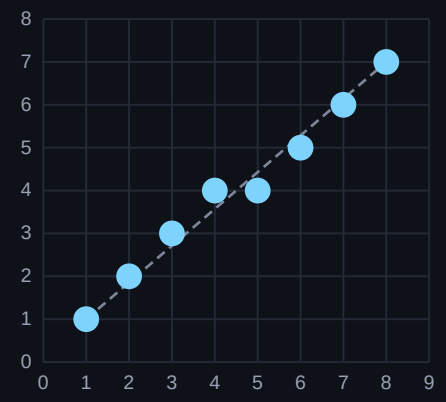
DDA problems.

(1, 1) → (8, 7)

X0 = 1, xEnd = 8, y0 = 1, yEnd = 7 · dx > dy · $x_{k+1} = x_k + 1$ · $y_{k+1} = y_k + m$

X	Y	(X, Y)
1	1	(1,1)
2	1.86	(2,2)
3	2.7	(3,3)
4	3.57	(4,4)
5	4.43	(5,4)
6	5.29	(6,5)
7	6.14	(7,6)
8	7	(8,7)

Plotted pixels



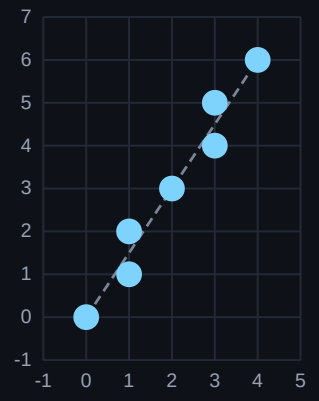
Lec 4 · p20

(0, 0) → (4, 6)

X0 = 0, xEnd = 4, y0 = 0, yEnd = 6 · dy > dx · $y_{k+1} = y_k + 1$ · $x_{k+1} = x_k + 1/m$

X	Y	(X, Y)
0	0	(0,0)
0.6	1	(1,1)
1.2	2	(1,2)
1.8	3	(2,3)
2.5	4	(3,4)
3.3	5	(3,5)
3.9	6	(4,6)

Plotted pixels



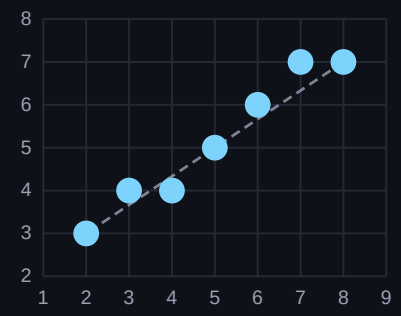
Lec 4 · p21

(9, 8) → (2, 3)

X0 = 9, xEnd = 2, y0 = 8, yEnd = 3 · m ≤ 1 · Steps = dx = (2 − 9) = −7 · k in range from 0 to 6 · $x_{k+1} = x_k - 1$ · $y_{k+1} = y_k - m$

K	x	y	(x, y)
0	8	7.2	(8,7)
1	7	6.5	(7,7)
2	6	5.8	(6,6)
3	5	5.1	(5,5)
4	4	4.4	(4,4)
5	3	3.7	(3,4)
6	2	3	(2,3)

Plotted pixels



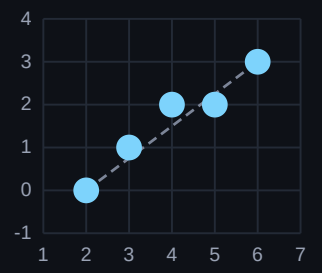
Lec 4 · p23

A(2, 0) → B(6, 3)

|Δx| = |6 − 2| = 4, |Δy| = |3 − 0| = 3 → |Δx| > |Δy| · $x_{k+1} = x_k + 1$, $y_{k+1} = y_k + m$ · m = 0.75

x	y	Setpixel(x, y)
2	0	(2,0)
3	0.75	(3,1)
4	1.5	(4,2)
5	2.25	(5,2)
6	3	(6,3)

Plotted pixels



Graphics sec5 · p1

(3, 2) → (7, 8)

$|\Delta x| = |7 - 3| = 4$, $|\Delta y| = |8 - 2| = 6 \rightarrow |\Delta y| > |\Delta x| \cdot x_{k+1} = x_k + 1/m$, $y_{k+1} = y_k + 1 \cdot 1/m = 0.667$

x	y	Setpixel(x, y)
3	2	(3,2)
3.667	3	(4,3)
4.33	4	(4,4)
5	5	(5,5)
5.66	6	(6,6)
6.33	7	(6,7)
7	8	(7,8)

Plotted pixels



Graphics sec5 · p1

الفكرة لو الخط مرسوم من اليمين للشمال، بتطرح 1 – x وm – y، أو 1 – y و1/m – x بدل ما تجمع (Lec 4 · p22).

T3

Source: Lec 4 · p25–32 · Graphics sec5 · p1

Bresenham's line algorithm.

- Accurate and efficient; uses only incremental integer calculations. The method is described for a line segment with a positive slope less than one, so we work by increment x and calculate y. The method generalizes to line segments of other slopes by considering the symmetry between the various octants and quadrants of the xy plane.

1. Input the two line endpoints and store the left endpoint in (x0, y0).
2. Load (x0, y0) into the frame buffer; that is, plot the first point.
3. Calculate constants Δx, Δy, 2Δy, and 2Δy − 2Δx, and obtain the starting value for the decision parameter: **p0 = 2Δy − Δx**.
4. At each x_k along the line, starting at k = 0: if $p_k < 0$, the next point to plot is $(x_k + 1, y_k)$ and **$p_{k+1} = p_k + 2\Delta y$**; otherwise, the next point to plot is $(x_k + 1, y_k + 1)$ and **$p_{k+1} = p_k + 2\Delta y - 2\Delta x$**.
5. Repeat step 4 Δx − 1 times.

- **Advantages (sec5):** significantly faster due to integer-only operations; more accurate and efficient, without round-off errors.
- **Disadvantages (sec5):** a full implementation must handle all combinations of slope and endpoint order by adjusting step sizes and interchanging X and Y coordinates, adding complexity.

- For |Δy| > |Δx|: swap x points with y points (sec5: "requiring coordinate swapping"); for negative slopes: coordinate decrements.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Write Bresenham's line algorithm.

1. Input the endpoints, plot (x0, y0).
2. Calculate Δx, Δy, 2Δy, 2Δy − 2Δx; **p0 = 2Δy − Δx**.
3. $p_k < 0 \rightarrow (x_k + 1, y_k)$, p + 2Δy · otherwise $(x_k + 1, y_k + 1)$, p + 2Δy − 2Δx.
4. Repeat Δx − 1 times.

الفكرة p0 = 2 Δy − Δx. لو p < 0 ← p + 2Δy، وغير كده p + 2Δy − 2Δx. ولو |Δy| > |Δx| اعمل swap لـ x وy.

T3b

Source: Lec 4 · p26–27, p31

Bresenham's approach and generalization.

- Draw a line from (10, 11) to (14, 13): decide what is the next pixel position, (11, 11) or (11, 12). For the pixel position $x_{k+1} = x_k + 1$, which one we should choose: $(x_k + 1, y_k)$ or $(x_k + 1, y_k + 1)$.
- Bresenham's algorithm is generalized to lines with arbitrary slope by considering the symmetry between the various octants and quadrants of the xy plane.
- For a line with positive slope greater than 1.0 (Δy > Δx), we interchange the roles of the x and y directions: we step along the y direction in unit steps and calculate successive x values nearest the line path.
- If the initial position for a line with positive slope is the right endpoint, both x and y decrease as we step from right to left. To plot the same pixels regardless of the starting endpoint, we always choose the upper (or the lower) of the two candidate pixels whenever $d_{lower} = d_{upper}$.
- For negative slopes, one coordinate decreases as the other increases.
- Special cases: horizontal lines (Δy = 0), vertical lines (Δx = 0) and diagonal lines (|Δx| = |Δy|) can each be loaded directly into the frame buffer without the line-plotting algorithm.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: How is Bresenham generalized?

- Slope > 1: interchange the roles of x and y.
- Negative slope: one coordinate decreases as the other increases.
- Horizontal, vertical and diagonal lines are loaded directly into the frame buffer.

P2

Source:

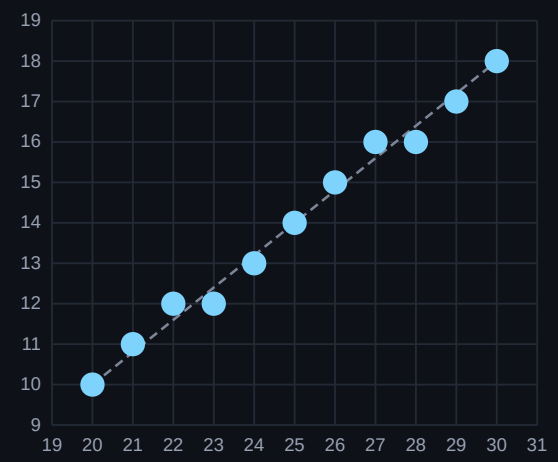
Bresenham problems.

(20, 10) → (30, 18)

$|\Delta x| = |30 - 20| = 10$, $|\Delta y| = |18 - 10| = 8$ · $|\Delta x| > |\Delta y|$ · $p0 = 2\Delta y - \Delta x = 16 - 10 = 6$ · $P_k \geq 0$: x and y incremented, $P_{k+1} = P_k - 4$ · $P_k < 0$: x incremented, y remains, $P_{k+1} = P_k + 16$

P	X	Y	SetPixel(x, y)
-	20	10	(20,10)
6	21	11	(21,11)
2	22	12	(22,12)
-2	23	12	(23,12)
14	24	13	(24,13)
10	25	14	(25,14)
6	26	15	(26,15)
2	27	16	(27,16)
-2	28	16	(28,16)
14	29	17	(29,17)
10	30	18	(30,18)

Plotted pixels



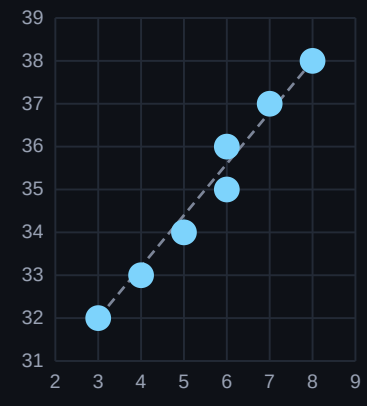
Lec 4 · p30

(3, 32) → (8, 38)

$|\Delta x| = |8 - 3| = 5$, $|\Delta y| = |38 - 32| = 6$ · $|\Delta y| > |\Delta x|$ · Swap x points with y points: X0 = 32, xend = 38 & y0 = 3, yend = 8 · $|\Delta x| = 6$, $|\Delta y| = 5$ · $p0 = 2\Delta y - \Delta x = 10 - 6 = 4$ · $P_k \geq 0$: $P_{k+1} = P_k - 2$ · $P_k < 0$: $P_{k+1} = P_k + 10$

P	X	Y	SetPixel(y, x)
-	32	3	(3,32)
4	33	4	(4,33)
2	34	5	(5,34)
0	35	6	(6,35)
-2	36	6	(6,36)
8	37	7	(7,37)
-	38	8	(8,38)

Plotted pixels



Lec 4 · p33

(8, 32) → (3, 38)

$|\Delta x| = |3 - 8| = |-5|$, $|\Delta y| = |38 - 32| = 6 \cdot |\Delta y| > |\Delta x| \cdot$ Swap: X0 = 32, xend = 38 & y0 = 8, yend = 3 · p0 = $2\Delta y - \Delta x = 10 - 6 = 4 \cdot P_k < 0$: X incremented and Y remain, $P_{k+1} = P_k + 10 \cdot P_k \geq 0$: X incremented and Y decremented, $P_{k+1} = P_k - 2$

P	X	Y	SetPixel(y, x)
-	32	8	(8,32)
4	33	7	(7,33)
2	34	6	(6,34)
0	35	5	(5,35)
-2	36	5	(5,36)
8	37	4	(4,37)
-	38	3	(3,38)

Plotted pixels



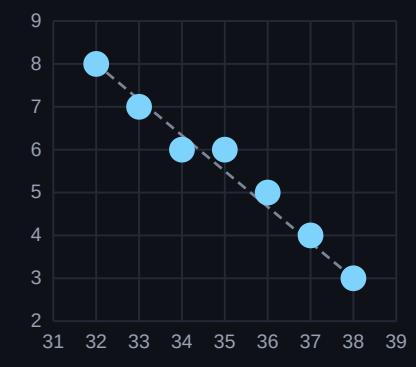
Lec 4 · p34

(38, 3) → (32, 8)

$|\Delta x| = |32 - 38| = 6$, $|\Delta y| = |8 - 3| = 5 \cdot |\Delta x| > |\Delta y| \cdot$ p0 = $2\Delta y - \Delta x = 10 - 6 = 4 \cdot P_k \geq 0$: X is decremented, Y is increment, $P_{k+1} = P_k - 2 \cdot P_k < 0$: X is decremented, Y is remain, $P_{k+1} = P_k + 10$

P	X	Y	SetPixel(x, y)
-	38	3	(38,3)
4	37	4	(37,4)
2	36	5	(36,5)
0	35	6	(35,6)
-2	34	6	(34,6)
8	33	7	(33,7)
-	32	8	(38,8)

Plotted pixels



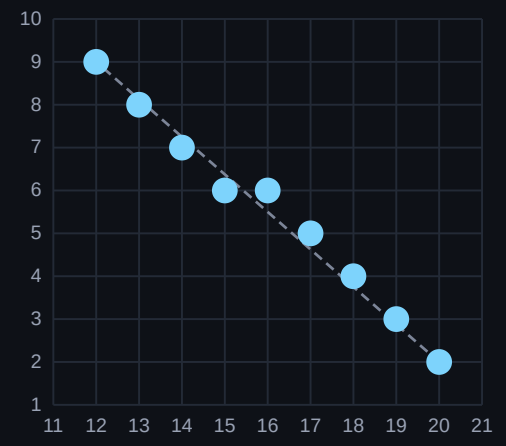
Lec 4 · p35

(20, 2) → (12, 9)

$|\Delta x| = |12 - 20| = |-8|$, $|\Delta y| = |9 - 2| = |7|$ · $|\Delta x| > |\Delta y|$ · $p0 = 2\Delta y - \Delta x = 14 - 8 = 6$ · If $p_k < 0$: Setpixel($x_k - 1$, y_k), $p_{k+1} = p_k + 14$ · Otherwise: Setpixel($x_k - 1$, $y_k + 1$), $p_{k+1} = p_k - 2$

P	x	y	Setpixel(x, y)
-	20	2	(20,2)
6	19	3	(19,3)
4	18	4	(18,4)
2	17	5	(17,5)
0	16	6	(16,6)
-2	15	6	(15,6)
12	14	7	(14,7)
10	13	8	(13,8)
8	12	9	(12,9)

Plotted pixels



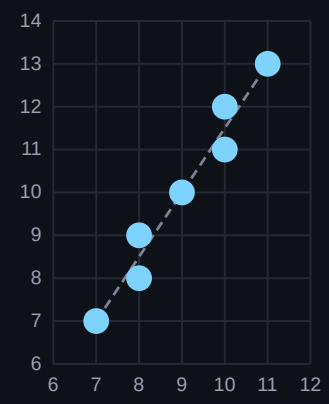
Graphics sec5 · p2

(7, 7) → (11, 13)

$|\Delta x| = |11 - 7| = |4|$, $|\Delta y| = |13 - 7| = |6|$ · $|\Delta y| > |\Delta x|$ · Swap x points with y points: X0 = 7, xend = 13 & y0 = 7, yend = 11 · $|\Delta x| = 6$, $|\Delta y| = 4$ · $p0 = 2\Delta y - \Delta x = 2$ · If $p_k < 0$: Setpixel($y_k + 1$, x_k), $p_{k+1} = p_k + 8$ · Otherwise: Setpixel($y_k + 1$, $x_k + 1$), $p_{k+1} = p_k - 4$

P	X	y	Setpixel(y, x)
-	7	7	(7,7)
2	8	8	(8,8)
-2	9	8	(8,9)
6	10	9	(9,10)
2	11	10	(10,11)
-2	12	10	(10,12)
6	13	11	(11,13)

Plotted pixels



Graphics sec5 · p2

T4

Source: Lec 4 · p36

Difference between DDA and Bresenham line drawing algorithms.

DDA	Bresenham

Arithmetic	Uses floating points, i.e. real arithmetic	Uses fixed points, i.e. integer arithmetic
Operations	Uses multiplication and division in its operations	Uses only subtraction and addition in its operations
Speed	Rather slowly than Bresenham because it uses real arithmetic (floating-point operations)	Faster than DDA because it performs only addition and subtraction and uses only integer arithmetic
Accuracy & efficiency	Not as accurate and efficient as Bresenham	More efficient and much accurate than DDA
Drawing	Can draw circles and curves but not as accurate as Bresenham	Can draw circles and curves with much more accuracy than DDA
Round off	Rounds off the coordinates to integer that is nearest to the line	Does not round off but takes the incremental value in its operation
Expensive	Uses an enormous number of floating-point multiplications so it is expensive	Less expensive than DDA as it uses only addition and subtraction

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: Main points to write.

- DDA: real arithmetic, multiplication and division, slower, rounds off, expensive.
- Bresenham: integer arithmetic, only addition and subtraction, faster, more accurate, less expensive.

CG / LECTURE 5 · SEC 6

Circle, Ellipse & Antialiasing

05

Midpoint circle · midpoint ellipse · aliasing and super-sampling

T1

Source: Lec 5 · p3–8

Drawing circle.

- A circle is a set of all points in a plane that are all an equal distance from a single point, the center. The distance from a circle's center to a point on the circle is called the radius of the circle.
- A unit circle is the circle of radius 1 centered at the origin (0, 0) in the Cartesian coordinate system.
- Pythagoras: $x^2 + y^2 = r^2$; for a unit circle $x^2 + y^2 = 1 \rightarrow y = \sqrt{(1 - x^2)}$. We could draw a quarter circle by incrementing x from 0 to R in unit steps and solving for +y. For another circle: $y = yc + \sqrt{(r^2 - (x - xc)^2)}$. Method needs lots of computation, and gives non-uniform pixel spacing.
- Trigonometry: $\sin \theta = y/r$, $\cos \theta = x/r \rightarrow x = r \cos \theta$ and $y = r \sin \theta$; unit circle $x = \cos \theta$, $y = \sin \theta$; another circle: $y = yc + \sin \theta \cdot r$, $x = xc + \cos \theta \cdot r$.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Why is the direct (Pythagoras) circle method not used?

- $y = yc \pm \sqrt{(r^2 - (x - xc)^2)}$ needs lots of computation and gives **non-uniform pixel spacing**.
- Trigonometric: $x = xc + r \cos \theta$, $y = yc + r \sin \theta$.

الفكرة الطرق المباشرة (Pythagoras و sin/cos) محتاجة حسابات كتير، والـ pixels بتطلع مش متساوية المسافات.

T2

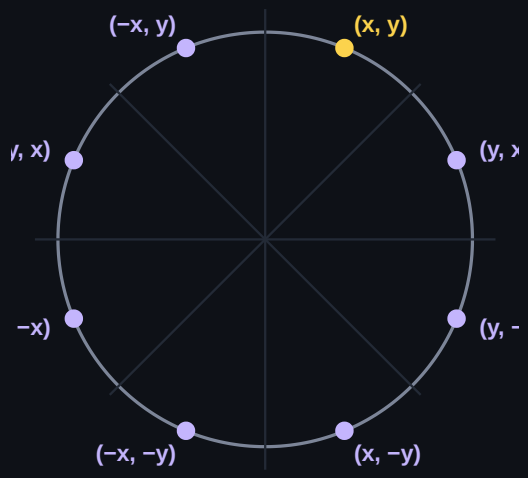
Source: Lec 5 · p9–13 · Graphics_Sec 6 · p1

Midpoint circle algorithm.

- $R^2 = x^2 + y^2 \rightarrow x^2 + y^2 - R^2 = 0 \rightarrow$ decision parameter $P = x^2 + y^2 - R^2$: $P = 0$ on the circle, $P > 0$ outside the circle, $P < 0$ inside the circle.
- Symmetry of a circle: calculation of a circle point (x, y) in one octant yields the circle points shown for the other seven octants.
- Initial point (0, r) & $P0 = 1 - r$.
- If $p_k < 0$ the midpoint is inside the circle: $x_{k+1} = x_k + 1$, $y_{k+1} = y_k$, $P_{k+1} = P_k + 2x_{k+1} + 1$.

Eight-way symmetry of a circle

- If $p_k > 0$ the midpoint is outside the circle: $x_{k+1} = x_k + 1$, $y_{k+1} = y_k - 1$, $P_{k+1} = P_k + 2x_{k+1} + 1 - 2y_{k+1}$.
- Setpixel(x, y) → Setpixel(xc + x, yc + y) when the center is not (0, 0).



✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Explain the midpoint circle algorithm.

- $P = x^2 + y^2 - R^2$: 0 on, > 0 outside, < 0 inside.
- Start (0, r), **P0 = 1 − r**.
- $P < 0$ → $(x + 1, y)$, $P + 2x_{k+1} + 1$ · $P > 0$ → $(x + 1, y - 1)$, $P + 2x_{k+1} + 1 - 2y_{k+1}$.
- One octant only; the other seven by eight-way symmetry; stop when $x \geq y$.

الفكرة P0 = 1 − r. بنحسب octant واحد بس، والباقي بالـ symmetry، وبنقف لما x ≤ y.

T2b

Source: Lec 5 · p11

Midpoint circle algorithm — steps.

1. Input radius r and circle center (x_c, y_c), and obtain the first point on the circumference of a circle centered on the origin as $(x_0, y_0) = (0, r)$.
2. Calculate the initial value of the decision parameter as $p_0 = 5/4 - r$.
3. At each x_k position, starting at k = 0: if $p_k < 0$, the next point is (x_{k+1}, y_k) and $p_{k+1} = p_k + 2x_{k+1} + 1$; otherwise the next point is $(x_k + 1, y_k - 1)$ and $p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$, where $2x_{k+1} = 2x_k + 2$ and $2y_{k+1} = 2y_k - 2$.
4. Determine symmetry points in the other seven octants.
5. Move each calculated pixel position (x, y) onto the circular path centered on (x_c, y_c): $x = x + x_c$, $y = y + y_c$.
6. Repeat steps 3 through 5 until $x \geq y$.

The lecture examples use P0 = 1 − r (p12, Sec 6).

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Steps (as in the lecture).

- Input r and center → (0, r) · p0 = 5/4 − r · test pk · symmetry in seven octants · move to (xc, yc) · repeat until x ≥ y.

P1

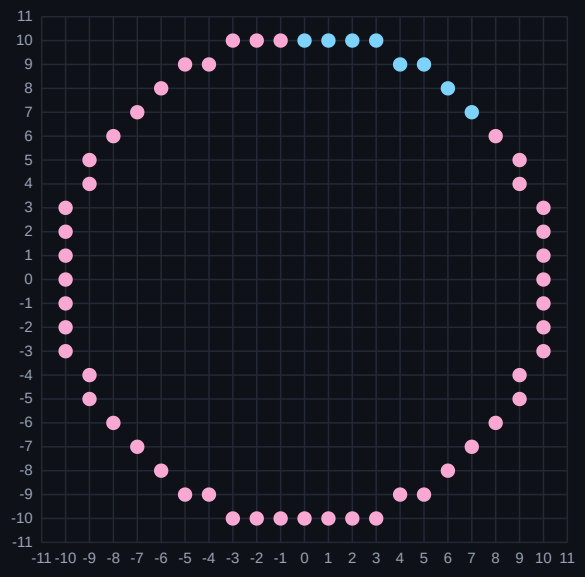
Source: Lec 5 · p12–13

Given a circle radius r = 10, we demonstrate the midpoint circle algorithm by determining positions along the circle octant in the first quadrant from x = 0 to x = y.

Initial point (x0, y0) → (0, 10)

P	X	Y	$2x_{k+1}$	$2y_{k+1}$
-	0	10	-	-
-9	1	10	2	20
-6	2	10	4	20
-1	3	10	6	20
6	4	9	8	18
-3	5	9	10	18
8	6	8	12	16
5	7	7	-	-

Octant points (blue) + eight-way symmetry (green)



Eight-way symmetry of (1, 10): Setpixel(1,10), (10,1), (10,−1), (1,−10), (−1,−10), (−10,−1), (−10,1), (−1,10).

P2

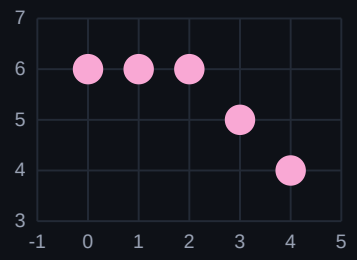
Source: Graphics_Sec 6 · p2

Using the mid-point circle drawing algorithm, draw a circle with a radius of 6 units, along the circle octant in the 1st quadrant.

Initial point (0, r) & P0 = 1 − r

p_k	x	Y
-	0	6
-5	1	6
-2	2	6
3	3	5
0	4	4

Octant points



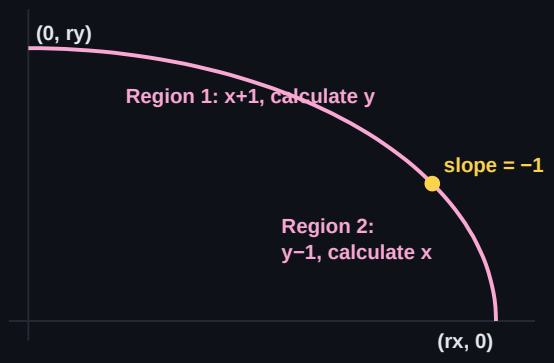
T3

Source: Lec 5 · p15–21 · Graphics_Sec 6 · p1

Ellipse-generating algorithm.

- Region 1: increment x, calculate y. Region 2: decrement y, calculate x.
- **Region 1:** initial point (x0, y0) = (0, r_y) & P0 = $r_y^2 - r_x^2 r_y + \frac{1}{4}r_x^2$. If $p_k < 0$ (inside): $x_{k+1} = x_k + 1$, $y_{k+1} = y_k$, $P_{k+1} = p_k + 2r_y^2 x_{k+1} + r_y^2$. If $p_k > 0$ (outside): $x_{k+1} = x_k + 1$, $y_{k+1} = y_k - 1$, $P_{k+1} = p_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$. Repeat until $2r_y^2 x \geq 2r_x^2 y$.
- **Region 2:** initial point = last point of region 1 & P0 = $r_y^2(x0 + \frac{1}{2})^2 + r_x^2(y0 - 1)^2 - r_x^2 r_y^2$. If $p_k < 0$ (inside): $X_{k+1} = x_k + 1$, $Y_{k+1} = y_k - 1$, $P_{k+1} = p_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$. If $p_k > 0$ (outside): $X_{k+1} = x_k$, $Y_{k+1} = y_k - 1$, $P_{k+1} = p_k - 2r_x^2 y_{k+1} + r_x^2$. Repeat until we reach point (r_x, 0).

Ellipse regions (first quadrant)



✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Explain the midpoint ellipse algorithm.

- Region 1: increment x, calculate y; start (0, ry), $p1_0 = ry^2 - rx^2 ry + \frac{1}{4}rx^2$; until $2ry^2 x \geq 2rx^2 y$.
- Region 2: decrement y, calculate x; start at the last point of region 1, $p2_0 = ry^2(x0 + \frac{1}{2})^2 + rx^2(y0 - 1)^2 - rx^2 ry^2$; until y = 0.

الفكرة Region 1: بنزوّد x. Region 2: بنقلّل y. وبننقل من region 1 لـ region 2 لما $2ry^2x \le 2rx^2y$.

T3b

Source: Lec 5 · p15–20

Ellipse: axes, symmetry, regions and algorithm steps.

- Figure 17: ellipse centered at (x_c, y_c) with semimajor axis r_y and semiminor axis r_x.
- Figure 19 — symmetry of an ellipse: calculation of a point (x, y) in one quadrant yields the ellipse points shown for the other three quadrants: (−x, y), (−x, −y), (x, −y).
- Ellipse processing regions: over region 1 the magnitude of the ellipse slope is less than 1.0; over region 2 the magnitude of the slope is greater than 1.0.

1. Input r_x, r_y and ellipse center (x_c, y_c), and obtain the first point on an ellipse centered on the origin as $(x_0, y_0) = (0, r_y)$.
2. Calculate the initial value of the decision parameter in region 1: $p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$.
3. At each x_k in region 1: if $p1_k < 0$ the next point is (x_{k+1}, y_k) and $p1_{k+1} = p1_k + 2r_y^2 x_{k+1} + r_y^2$; otherwise $(x_k + 1, y_k - 1)$ and $p1_{k+1} = p1_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$, with $2r_y^2 x_{k+1} = 2r_y^2 x_k + 2r_y^2$, $2r_x^2 y_{k+1} = 2r_x^2 y_k - 2r_x^2$; continue until $2r_y^2 x \ge 2r_x^2 y$.
4. Calculate the initial value of the decision parameter in region 2: $p2_0 = r_y^2(x_0 + \frac{1}{2})^2 + r_x^2(y_0 - 1)^2 - r_x^2 r_y^2$, where (x_0, y_0) is the last position calculated in region 1.
5. At each y_k in region 2: if $p2_k > 0$ the next point is $(x_k, y_k - 1)$ and $p2_{k+1} = p2_k - 2r_x^2 y_{k+1} + r_x^2$; otherwise $(x_k + 1, y_k - 1)$ and $p2_{k+1} = p2_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$; continue until $y = 0$.
6. For both regions, determine symmetry points in the other three quadrants.
7. Move each calculated pixel position (x, y) onto the elliptical path centered on (x_c, y_c): $x = x + x_c$, $y = y + y_c$.

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: Ellipse: axes, symmetry, regions.

- Semimajor axis ry, semiminor axis rx.
- A point (x, y) in one quadrant gives (−x, y), (−x, −y), (x, −y).
- Region 1: |slope| < 1 · Region 2: |slope| > 1.

P3

Source: Lec 5 · p22–23

Midpoint ellipse drawing: rx = 8, ry = 6.

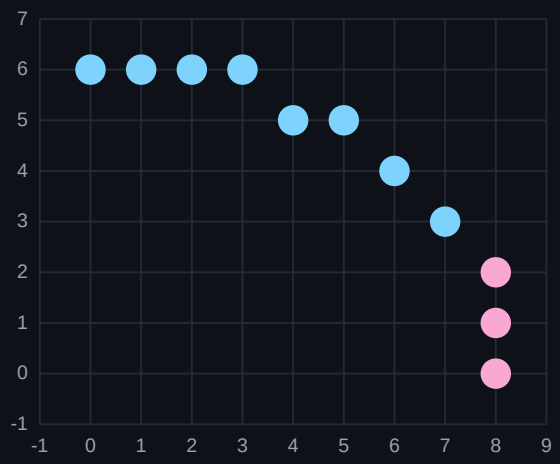
Region 1 · initial point (0, 6)

k	$p1_k$	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	−332	(1, 6)	72	768
1	−224	(2, 6)	144	768
2	−44	(3, 6)	216	768
3	208	(4, 5)	288	640
4	−108	(5, 5)	360	640
5	288	(6, 4)	432	512
6	244	(7, 3)	504	384

Region 2 · initial point = last point of region 1 (7, 3)

k	$p2_k$	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	−151	(8, 2)	576	256
1	233	(8, 1)	576	128
2	745	(8, 0)	—	—

Region 1 (blue) and region 2 (green)



P4

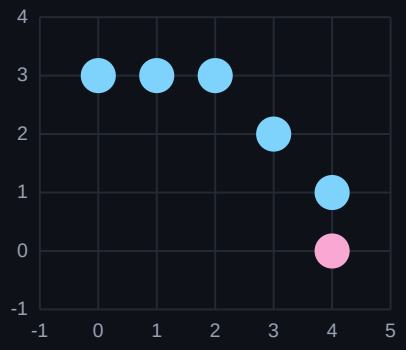
Source: Graphics_Sec 6 · p2

Determine pixel positions along the ellipse path in the first quadrant using midpoint ellipse algorithm: rx = 4, ry = 3.

Pk_region1	x	y	$2r_y^2x$	$2r_x^2y$
	0	3	0	96
−35	1	3	18	96
−8	2	3	36	96
37	3	2	54	64
36	4	1	72	32

Pk_region2	x	y	$2r_y^2x$	$2r_x^2y$
	4	1	72	32
38	4	0		

Region 1 (blue) and region 2 (green)



T4

Source: Lec 5 · p24–25

Implementation methods for antialiasing.

- Line segments and other graphics primitives generated by the raster algorithms have a jagged, or stair-step, appearance. This distortion of information due to low-frequency sampling (under sampling) is called **aliasing**.
- We can improve the appearance of displayed raster lines by applying **antialiasing** methods that compensate for the under sampling process.
- One way to increase sampling rate with raster systems is simply to display objects at higher resolution. However, even at the highest resolution possible with current technology, the jaggies will be apparent to some extent. There is a limit to how big we can make the frame buffer and still maintain the refresh rate at 60 frames or more per second.
- A straightforward antialiasing method is to increase sampling rate by treating the screen as if it were covered with a finer grid than is actually available, using multiple sample points across this finer grid to determine an appropriate intensity level for each screen pixel. This technique is called **super-sampling** (or post-filtering).

- Supersampling algorithms are often implemented by giving more weight to subpixels near the center of a pixel area.

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: Define aliasing and antialiasing; explain super-sampling.

- **Aliasing:** the jagged (stair-step) appearance — distortion due to under sampling.
- **Antialiasing:** methods that compensate for the under sampling process.
- **Super-sampling (postfiltering):** treat the screen as covered with a finer grid and use multiple sample points to get each pixel's intensity; more weight to subpixels near the center.
- Higher resolution alone is not enough: the jaggies remain, and there is a limit to the frame buffer size at 60 fps.

الفكرة الـ aliasing هو الحواف المسننة. والـ super-sampling بيعالجه بـ grid أدق من الشاشة.

CG / LECTURE 6 · SEC 7

Geometric Transformations

06

2D / 3D transformations · homogeneous coordinates · composite transformations

T1

Source: Lec 6 · p2–5 · Graphics_Sec 7 · p1

Geometric transformation.

- Geometric transformation functions: translation, scaling, rotation, reflection, shearing.
- Geometric transformations map points in one space to points in another. These transformations can be linear or non-linear transformations.
- We'll focus on linear transformations that preserve co-linearity and ratios of distances (parallel lines remain parallel): maps origin to origin; **co-linearity:** all points lying on a line initially still lie on a line after transformation; **ratios of distances:** the midpoint of a line segment remains the midpoint after transformation.
- Linear transformation can be represented by (non-singular) matrices: [x′ y′] = M·[x y], M a square matrix of n columns and n rows; M = [a b; c d] → x′ = ax + by & y′ = cx + dy.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Define the linear transformation and its properties.

- Geometric transformations map points in one space to points in another.
- Linear transformations preserve **co-linearity** and **ratios of distances** (parallel lines remain parallel).
- Maps origin to origin · points on a line stay on a line · the midpoint stays the midpoint.
- Represented by a non-singular square matrix: x′ = ax + by, y′ = cx + dy.

الفكرة الـ linear transformation بتحافظ على الـ co-linearity والـ ratios of distances، والخطوط المتوازية بتفضل متوازية.

T2

Source: Lec 6 · p6–12

2D scaling, reflection, rotation, shear and translation.

- **Scaling from the origin:** x′ = sx·x, y′ = sy·y; S = [sx 0; 0 sy]. Properties: doesn't preserve angles between lines except with uniform scaling sx = sy; the object must be centered on origin, or it will be shifted.
- **Reflection** creates a mirror image of an object across a line or point. Horizontal reflection: sy = −1 → x′ = x, y′ = −y. Vertical reflection: sx = −1 → x′ = −x, y′ = y. Diagonal reflection: sx = −1 and sy = −1 → x′ = −x, y′ = −y.
- **Rotation about the origin:** x = r cos φ, y = r sin φ, x′ = r cos(θ + φ), y′ = r sin(θ + φ) → **x′ = x cos θ − y sin θ, y′ = x sin θ + y cos θ**; R = [cos θ −sin θ; sin θ cos θ]; P′ = R·P.

- **Shear (skewing)** deforms shape like shifted slices. Shear in x: x′ = x + shx·y, y′ = y. Shear in y: x′ = x, y′ = shy·x + y.
- **Translation** over vector (tx, ty): x′ = x + tx, y′ = y + ty, or P′ = P + T.

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: Write the 2D transformations.

Transformation	Equations
Scaling	x′ = sx·x, y′ = sy·y
Reflection	horizontal y′ = −y · vertical x′ = −x · diagonal both
Rotation	x′ = x cos θ − y sin θ, y′ = x sin θ + y cos θ
Shear	x: x′ = x + shx·y · y: y′ = shy·x + y
Translation	x′ = x + tx, y′ = y + ty

Q: Properties of scaling.

- Doesn't preserve angles except with uniform scaling sx = sy.
- The object must be centered on origin, or it will be shifted.

الفكرة Horizontal reflection ← y بالسالب. Vertical reflection ← x بالسالب. Diagonal ← الاتنين بالسالب.

T3

Source: Lec 6 · p13–19 · Graphics_Sec 7 · p1

Homogeneous coordinates.

- Scale P′ = S·P, rotation P′ = R·P, shear P′ = Shr·P, translation P′ = T + P. We would like all transformations to be multiplications so we can concatenate them to combine transformations.
- Homogeneous coordinates: uniform representation of translation, rotation, scaling; uniform representation of points and vectors; compact representation of a sequence of transformations. Add an extra coordinate, W (weight = 1), to a point: P(x, y) → P(x, y, 1).
- Transformation matrices for 2D translation are now 3 × 3.

Transformation	2D Matrix	Inverse 2D Matrix
Translation	$\begin{vmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{vmatrix}$	$\begin{vmatrix} 1 & 0 & -tx \\ 0 & 1 & -ty \\ 0 & 0 & 1 \end{vmatrix}$
Scaling	$\begin{vmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{vmatrix}$	$\begin{vmatrix} 1/sx & 0 & 0 \\ 0 & 1/sy & 0 \\ 0 & 0 & 1 \end{vmatrix}$

Reflection	horizontal $\begin{vmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$ vertical $\begin{vmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$	horizontal $\begin{vmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$ vertical $\begin{vmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$
Rotation	$R(\theta) = \begin{vmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{vmatrix}$	$R^T(\theta) = \begin{vmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{vmatrix}$
Shearing	in y $\begin{vmatrix} 1 & 0 & 0 \\ shy & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$ in x $\begin{vmatrix} 1 & shx & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$	in y $\begin{vmatrix} 1 & 0 & 0 \\ 1/shy & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$ in x $\begin{vmatrix} 1 & 1/shx & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$

Inverse rotation matrices: $R^{-1}(\theta) = R(-\theta) = R^T(\theta)$.

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: Why do we use homogeneous coordinates?

- Translation is an addition; we want **all transformations to be multiplications** so we can concatenate them.
- Uniform representation of translation, rotation, scaling and of points and vectors; compact sequence of transformations.
- Add W = 1: P(x, y) → P(x, y, 1).

الفكرة بنضيف W = 1 عشان كل الـ transformations تبقى ضرب، حتى الـ translation.

T4

Source: Lec 6 · p20–28

Concatenation and properties.

	Concatenation	Properties
Translation	P″ = P·T(tx1, ty1)·T(tx2, ty2) = P·T(tx1 + tx2, ty1 + ty2)	1. T(0, 0) = I · 2. T(tx1, ty1)·T(tx2, ty2) = T(tx1 + tx2, ty1 + ty2) · 3. T^{-1}(tx1, ty1) = T(−tx1, −ty1)
Scaling	S(sx1, sy1)·S(sx2, sy2) = S(sx1·sx2, sy1·sy2) — only diagonal elements in the matrix, easy to multiply	1. S(1, 1) = I · 2. S(sx1, sy1)·S(sx2, sy2) = S(sx1·sx2, sy1·sy2) · 3. S^{-1}(sx, sy) = S(1/sx, 1/sy)
Rotation	R(θ)·R(φ) = R(θ + φ)	R(0) = I · R(θ)·R(φ) = R(θ + φ) · R(θ)·R(φ) = R(φ)·R(θ) · inverse rotation $R^{-1}(\theta) = R^T(\theta)$

2D polygon transformation: translate polygon — apply the same operation on all points. Works always, for all transformations of objects defined as a set of points.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Properties to write.

- $T(t1)\cdot T(t2) = T(t1 + t2)$, $T^{-1} = T(-tx, -ty)$.
- $S(s1)\cdot S(s2) = S(sx1\cdot sx2, sy1\cdot sy2)$, $S^{-1} = S(1/sx, 1/sy)$.
- $R(\theta)\cdot R(\varphi) = R(\theta + \varphi) = R(\varphi)\cdot R(\theta)$, $R^{-1}(\theta) = R^{T}(\theta)$.

الفكرة الـ translations بتتجمع، والـ scales بتتضرب، وزوايا الـ rotations بتتجمع.

T5

Source: Lec 6 · p29–31

2D composite transformation.

- Combine transformations of different type: translate, rotate, translate; translate, scale, translate; translate, reflect, translate. Use to rotate or scale an object w.r.t. a point that is not the origin. Implement by multiplication of the corresponding homogeneous matrices.
- **Rotation around a pivot point:** 1. Translate the object so that the pivot point moves to the origin. 2. Rotate around origin. 3. Translate the object so that the pivot point is back to its original position. **$P' = T(tx, ty)\cdot R(\theta)\cdot T(-tx, -ty)\cdot p$**.
- **Scale around a pivot point:** same three steps → **$P' = T(tx, ty)\cdot S(sx, sy)\cdot T(-tx, -ty)\cdot p$**.

✍ EXAM ANSWER — WRITE THIS FOR THE FULL MARK

Q: Rotate (or scale) an object about a point that is not the origin.

1. Translate the object so that the pivot point moves to the origin.
2. Rotate (scale) around the origin.
3. Translate the pivot point back to its original position.
4. **$P' = T(tx, ty)\cdot R(\theta)\cdot T(-tx, -ty)\cdot P$** — read right to left.

الفكرة الدوران أو الـ scale حوالين نقطة: $T(p) \cdot R \cdot T(-p)$.

T6

Source: Lec 6 · p37–42 · Graphics_Sec 7 · p2

Transformation in 3D.

- Just add one more coordinate/axis; a point is represented as [x y z]. Remember to use homogeneous coordinates: embed scale and rotation matrices as upper-left sub-matrices and translation vectors as upper-right sub-vectors of the right column [x y z 1].
- Scaling looks just like the 2D version; we just added an sz term. Translation: similar to the 2D version, just with one more entry dz.
- Reflection in 3D: reflection over planes.

- Rotation in 3D: need to specify which axis the rotation is about; counterclockwise when looking along the positive half towards the origin.
- Inverses and composition in 3D: composition works exactly the same way.
- Scale an object w.r.t. a fixed position (xf, yf, zf): 1. translate the fixed point to the origin; 2. apply the scaling transformation relative to the coordinate origin; 3. translate the fixed point back to its original position.

Transformation	3D Matrix	Inverse 3D Matrix
Translation	$\begin{vmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{vmatrix}$	$\begin{vmatrix} 1 & 0 & 0 & -tx \\ 0 & 1 & 0 & -ty \\ 0 & 0 & 1 & -tz \\ 0 & 0 & 0 & 1 \end{vmatrix}$
Scaling	$\begin{vmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$	$\begin{vmatrix} 1/sx & 0 & 0 & 0 \\ 0 & 1/sy & 0 & 0 \\ 0 & 0 & 1/sz & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$
Reflection	yz plane $\begin{vmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$ xz plane $\begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$ xy plane $\begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$	the same matrices
Rotation	$Rx(\theta)$ $\begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$ $Ry(\theta)$ $\begin{vmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$ $Rz(\theta)$ $\begin{vmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$	$Rx^T(\theta)$ $\begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & \sin\theta & 0 \\ 0 & -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$ $Ry^T(\theta)$ $\begin{vmatrix} \cos\theta & 0 & -\sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$ $Rz^T(\theta)$ $\begin{vmatrix} \cos\theta & \sin\theta & 0 & 0 \\ -\sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$

✍ **EXAM ANSWER — WRITE THIS FOR THE FULL MARK**

Q: What changes in 3D?

- Points [x y z 1], 4 × 4 matrices.
- Reflection over planes.
- Rotation: specify the axis; counterclockwise looking towards the origin.
- Scale about a fixed point: translate to origin → scale → translate back.

الفكرة في 3D نفس الفكرة بمصفوفات 4 × 4، ولازم تحدد محور الدوران.

P1

Source: Lec 6 · p32

SOLVE: Rotate point p by angle 50 degree, then scale it to the double horizontally, then translate it 10 points to the bottom.

$$P' = R(50) \cdot P$$

$$P' = S(2,\ 1) \cdot R(50) \cdot p$$

$$P' = T(0,\ -10) \cdot S(2,\ 1) \cdot R(50) \cdot p$$

$$\begin{vmatrix} x' \\ y' \\ 1 \end{vmatrix} = \begin{vmatrix} 1 & 0 & 0 \\ 0 & 1 & -10 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 50 & -\sin 50 & 0 \\ \sin 50 & \cos 50 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} x \\ y \\ 1 \end{vmatrix}$$

الفكرة أول عملية بتتكتب جنب P، وآخر عملية بتتكتب على الشمال خالص.

P2

Source: Lec 6 · p33

Rotate point p by angle 40 degree around point (3, 4).

$$P' = T(-3,\ -4) \cdot p$$

$$P' = R(40) \cdot T(-3,\ -4) \cdot P$$

$$P' = T(3,\ 4) \cdot R(40) \cdot T(-3,\ -4) \cdot P$$

$$\begin{vmatrix} x' \\ y' \\ 1 \end{vmatrix} = \begin{vmatrix} 1 & 0 & 3 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 40 & -\sin 40 & 0 \\ \sin 40 & \cos 40 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -3 \\ 0 & 1 & -4 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} x \\ y \\ 1 \end{vmatrix}$$

P3

Source: Lec 6 · p34–36

Perform a rotation by angle 90 degree of a triangle with vertices A(0, 0), B(2, 1) and C(5, 4): about origin, about the point (−2, −1), about the triangle centroid.

- **About origin:** PA′ = R(90)·A, PB′ = R(90)·B, PC′ = R(90)·C.
- **About point (−2, −1):** PA′ = T(−2, −1)·R(90)·T(2, 1)·A (the same for B and C).
- **About the triangle centroid:** triangle centroid = ((0 + 2 + 5)/3, (0 + 1 + 4)/3) = (2.33, 1.66) → PA′ = T(2.33, 1.66)·R(90)·T(−2.33, −1.66)·A (the same for B and C).

$$PB' = \begin{vmatrix} 1 & 0 & -2 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 90 & -\sin 90 & 0 \\ \sin 90 & \cos 90 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 \\ 1 \\ 1 \end{vmatrix}$$

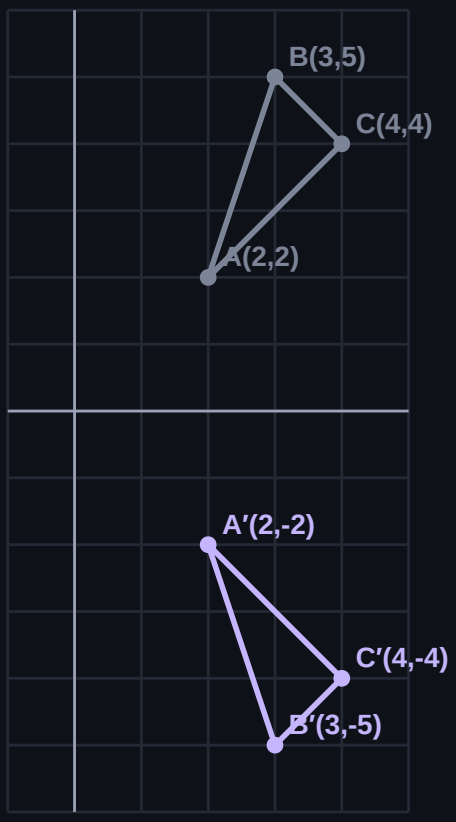
P4

Source: Graphics_Sec 7 · p3

A triangle is defined by three points A(2, 2), B(3, 5), C(4, 4): what's the transformation coordinate after horizontal reflection?

Answer: **A(2, −2), B(3, −5), C(4, −4)**

Before (gray) and after horizontal reflection (purple)



P5

Source: Graphics_Sec 7 · p3

Rotate point p by angle 50 degree, then rotate it by angle −60, then scale it to half vertically.

$$\begin{vmatrix} x' \\ y' \\ 1 \end{vmatrix} = \begin{vmatrix} 1 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos -10 & -\sin -10 & 0 \\ \sin -10 & \cos -10 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} x \\ y \\ 1 \end{vmatrix}$$

P6

Source: Graphics_Sec 7 · p3

Rotate point p by angle 20 around point (5, 6), then rotate angle by −50 around point (3, 7).

$$\begin{vmatrix} x' \\ y' \\ 1 \end{vmatrix} = \begin{vmatrix} 1 & 0 & 3 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 40 & -\sin 40 & 0 \\ \sin 40 & \cos 40 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 0 & 0 & 2 \\ 0 & 0 & -1 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 20 & -\sin 20 & 0 \\ \sin 20 & \cos 20 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 0 & 0 & -5 \\ 0 & 0 & -6 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} x \\ y \\ 1 \end{vmatrix}$$

P7

Source: Graphics_Sec 7 · p3

Magnify a triangle with vertices A(1, 1), B(2, 3) and C(7, 3) to twice its size while keeping C(7, 3) fixed.

$$PA' = \begin{vmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -7 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 \\ 1 \\ 1 \end{vmatrix}$$

$$PB' = \begin{vmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -7 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 \\ 3 \\ 1 \end{vmatrix}$$

$$PC' = \begin{vmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -7 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 7 \\ 3 \\ 1 \end{vmatrix}$$

P8

Source: Graphics_Sec 7 · p3

For a pyramid centred at (5, 2, 1): scale in object space by 2 in the x, y, and 3 in the z; rotate clockwise in object's space by 10° around x axis, 20° around y, and 70° around z; translate by (1, 7, 3) in world space.

$$\begin{vmatrix} 1 & 0 & 0 & 6 \\ 0 & 1 & 0 & 9 \\ 0 & 0 & 1 & 4 \\ 0 & 0 & 0 & 1 \end{vmatrix} \cdot Rz(70) \cdot Ry(20) \cdot Rx(10) \cdot \begin{vmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & 0 & -5 \\ 0 & 1 & 0 & -2 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

CG / LECTURES 3 – 5

OpenGL / GLUT Code

07

The code given in the lecture slides

C1

Source: Lec 3 · p10

Display-window management: main and myInit.

```
int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB);//set display mode
    glutInitWindowSize(400, 300);//set window size
    glutInitWindowPosition(100, 150);//set window position
    glutCreateWindow("graphics");
    myInit();
    glutDisplayFunc(myDisplay);
    glutReshapeFunc(reshape);
    Timer(0);
    glutMainLoop();
    return 0;
}
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200, 200, -150, 150);
}
```

C1b

Source: Lec 3 · p11

myDisplay with gluOrtho2D(−200, 200, −150, 150).

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT); // if it doesn't written there will be no display
    glPointSize(5);
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_TRIANGLES);
    glVertex2f(-200, 10);
    glVertex2f(10,10);
    glVertex2f(10, 150);
    glEnd();
    glFlush();
}
```

C1c

Source: Lec 3 · p17

Can I draw a simple figure without gluOrtho2D?

```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    //gluOrtho2D(-100,100,-100,100);
}
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_TRIANGLES);
    glVertex2f(-0.5, 0);
    glVertex2f(0, 0.5);
    glVertex2f(0.5, 0);
    glEnd();
    glFlush();
}
```

C2

Source: Lec 3 · p18

Create multiple viewports where each viewport has a different scene.

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glViewport(0, 0, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glViewport(100, 100, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glViewport(200, 200, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glFlush();
}
```

C3

Source: Lec 4 · p18

lineDDA.

```
inline int round (const float a) { return int (a + 0.5); }
void lineDDA (int x0, int y0, int xEnd, int yEnd)
{
    int dx = xEnd - x0, dy = yEnd - y0, steps, k;
    float xIncrement, yIncrement, x = x0, y = y0;
    if (fabs (dx) > fabs (dy))
        steps = fabs (dx);
    else
        steps = fabs (dy);
    xIncrement = float (dx) / float (steps); // xIncrement =1
    yIncrement = float (dy) / float (steps); //yIncrement =dy/dx=m
    setPixel (round (x), round (y));
    for (k = 0; k < steps; k++) {
        x += xIncrement;
        //x=x+1
        y += yIncrement;
        //y=y+m
        setPixel (round (x), round (y));
    }
}
```

C4

Source: Lec 5 · p14

Midpoint circle algorithm.

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT); // if it doesn't written there will be no display
    glPointSize(1);
    circleMidpoint(0, 0, 100,GL_POINTS);
    glFlush();
}
void circleMidpoint(int xc, int yc, int radius, GLenum mode)
{
    int x = 0;
    int y = radius;
    int p0 = 1 - radius;
    circleplot(xc, yc, x, y, mode);
    while (x < y)
    {
        x++;
        if (p0 < 0)
            p0 = p0 + 2 * x + 1;
        else
        {
            y--;
            p0 = p0 + 2 * x -2*y + 1;
        }
        circleplot(xc, yc, x, y, mode);
    }
}
void circleplot(int xc, int yc, int x, int y, GLenum mode)
{
    glBegin(mode);
    glVertex2f(xc + x, yc + y);
    glVertex2f(xc - x, yc + y);
    glVertex2f(xc + x, yc - y);
    glVertex2f(xc - x, yc - y);
    glVertex2f(xc + y, yc + x);
    glVertex2f(xc - y, yc + x);
    glVertex2f(xc + y, yc - x);
    glVertex2f(xc - y, yc - x);
    glEnd();
}
```

C5

Source: Lec 5 · p26

Write the implementation of an ellipse midpoint algorithm?

This question is asked at the end of Lec 5 (p26); no code for it is given in the references.

CG / EXERCISE SHEET 1

Exercise Sheet 1

08

Each question and where its answer is

E1

Source: Exercise Sheet 1

Exercise Sheet 1 — the answer of each question is in:

#	Question	Answer in
Q1-1	Relationship between computer graphics and image processing	Lec 1 · p6
Q1-2	Interactive drawing, input primitive (x, y)	Lec 1 · p13
Q1-3	In a raster display, the frame buffer is used to	Lec 1 · p15
Q1-4	A disadvantage of raster graphics compared to vector	Lec 1 · p15
Q1-5	Pipeline stage that creates pixels for each part of the object	Lec 1 · p24
Q1-6	NOT a component of the software level	Lec 1 · p17
Q2-1	Vector displays store line-drawing instructions	Lec 1 · p15
Q2-2	Raster graphics produce smaller file sizes	Lec 1 · p16
Q2-3	A valuator provides scalar values between 0 and 1	Lec 1 · p13
Q2-4	The frame buffer holds all pixels of a video frame	Lec 1 · p12
Q2-5	Transformations in 2D can be composed by multiplying matrices	Lec 1 · p23
Q2-6	The vertex shader is a fixed-function stage	Lec 1 · p24
Q2-7	Vector graphics are best for complex color blends	Lec 1 · p16
Q2-8	Clipping cuts vertices that will not appear on the screen	Lec 1 · p24
Q3-1	Define computer graphics and its three operations	Section 01 · T1 (Lec 1 · p7)
Q3-2	Geometric vs mathematical models	Section 01 · T3 (Lec 1 · p9)
Q3-3	Non-interactive vs interactive	Section 01 · T4 (Lec 1 · p11)
Q3-4	Role of the GPU	Section 01 · T5 (Lec 1 · p12)
Q3-5	Stages of the OpenGL pipeline	Section 01 · T10 (Lec 1 · p24)
Q3-6	Interdisciplinary nature of computer graphics	Section 01 · T2 (Lec 1 · p7)
Q3-7	Vector vs raster	Section 01 · T7 (Lec 1 · p15–16)

Q3-8 Vector or raster: logo, photo, icon — Section 01 · T7 (Lec 1 · p16)

CG / APPENDIX

Sources

A

Which part of this guide came from which reference file

File	Used in	Pages
`Computer Graphics and Man-Machine Interface_Lec 1.pdf`	Section 01 (T1–T11) · Exercise Sheet answers	p6–p26
`…_Lec 2.pdf`	Section 02 (T1–T7b, P1)	p3–p39
`…_Lec 3.pdf`	Section 03 (T1, T3, T3b, T4–T6, P1) · Code C1, C1b, C1c, C2	p2–p27
`…_Lec 4.pdf`	Section 04 (T1–T4, DDA and Bresenham tables) · Code C3	p3–p36
`…_Lec 5.pdf`	Section 05 (T1–T4, circle r = 10, ellipse 8 × 6) · Code C4, C5	p3–p26
`…_Lec 6.pdf`	Section 06 (T1–T6, P1–P3)	p2–p42
`SEC 1 and 2.pdf`	Section 01 (T1, T3, T4, T5, T6, T7, T8, T10)	p1–p4
`Graphics_SEC 3.pdf`	Section 02 (T1, T3, T4, T7, P1, P2, P3)	p1–p2
`Graphics_Sec 4.pdf`	Section 03 (T1, T2, T3, P1, T6)	p1–p2
`Graphics sec5.pdf`	Section 04 (T2, T3, DDA A(2,0)→B(6,3), (3,2)→(7,8), Bresenham (20,2)→(12,9), (7,7)→(11,13))	p1–p2
`Graphics_Sec 6.pdf`	Section 05 (T2, T3, circle r = 6, ellipse 4 × 3)	p1–p2
`Graphics_Sec 7.pdf`	Section 06 (T1, T3, T6, P4–P8)	p1–p3
`Exercise Sheet 1.pdf`	Section 08 (questions)	p1–p2

Nothing in this guide comes from any other file or from outside these 13 files.